457.557 Water Resources Systems Engineering, Spring 2022

In-class Exercise (Topic: ANN & Multi-objective Visual Analytics)

1. Consider the following example of ANN and solve in R:

   A. Consider the following example provided in the Lecture Note 05:

Technical Knowledge Score (TKS)	Communication Skills Score (CSS)	Student Placed
20	90	Placed
10	20	Not Placed
30	40	Not Placed
20	50	Not Placed
80	50	Placed
30	80	Placed

   i. First, install the package named “neuralnet” and load it on your R project.

   ii. Before creating the dataset for training & testing, set the seed number as 42. Create a data frame named “df” by combining TKS, CSS, and Placed.

   iii. Build a neural network model with logistic activation function with 3 hidden layers. Your inputs should be TKS and CSS, and your outputs should be a binary data (either placed or not placed). Then, you plot the network.

   iv. Consider the flowing test set. You will use the ANN model you built in the previous step to predict the following test data set.

Technical Knowledge Score (TKS)	Communication Skills Score (CSS)	Student Placed
20	90	?
10	20	?
30	40	?

   v. Create the data frame for the test data set named “test” and predict using the following commands: predict=compute(nn,test). Check your results using the command “predict$net.result.”

   vi. Finally, convert the probability into binary classes by setting the threshold level as 0.5. Use “ifelse” function to conduct this task.

B. Consider the following ANN example and solve it in R. Prior to solving the problem, you should include the 'cereals.csv' file into the same directory in which your R project is located:

i. First, install the package named "neuralnet" and load it on your R project.

ii. Read the 'cereals.csv' data and save it as 'data' by the following command: data = read.csv("cereals.csv", header=T).

iii. You will sample 60% of the original data and use it as the training data set, and the remaining 40% as the test data set. Conduct the following commands to sample & create train and test sets:

```
samplesize = 0.60 * nrow(data)
set.seed(80)
index = sample( seq_len ( nrow ( data ) ), size = samplesize )

datatrain = data[ index, ]
datatest = data[ -index, ]
```

iv. Since the data is not scaled, you will use the 'min-max normalization' method to scale your data into scales between 0 to 1 (minimum value of the data is transformed into 0, and the maximum value of the data is transformed into 1. Conduct the following commands to transform your data.

```
max = apply(data , 2 , max)
min = apply(data, 2 , min)
scaled = as.data.frame(scale(data, center = min, scale = max - min))
```

v. Now, you will fit your training data into an ANN model with logistic activation function with 3 hidden layers. Your inputs will be the first five variables (calories, protein, fat, sodium, and fiber) and your output will be 'rating.' Plot your ANN model.

```
trainNN = scaled[index , ]
testNN = scaled[-index , ]

set.seed(2)
NN = neuralnet(rating ~ calories + protein + fat + sodium + fiber,
  trainNN, hidden = 3, act.fct = "logistic", linear.output = T )
```

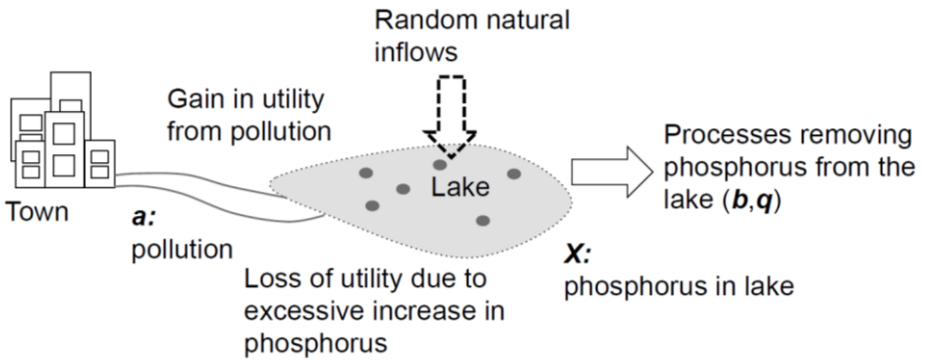
vi. Finally, test your model with the test data set and plot the predicted results with actual rating values. Compute the RMSE of your predicted results:

```
predict_testNN = compute(NN, testNN[,c(1:5)])
predict_testNN = (predict_testNN$net.result *
(max(data$rating) - min(data$rating))) + min(data$rating)

plot(datatest$rating, predict_testNN, col='blue', pch=16, ylab =
"predicted rating NN", xlab = "real rating") abline(0,1)

RMSE.NN = (sum((datatest$rating - predict_testNN)^2) / nrow(datatest)) ^ 0.5
```

2. In this problem, we'll use J3, a free desktop application for producing and sharing high-dimensional, interactive scientific visualizations. To download J3, follow the instructions on the last page of this assignment.

   We'll explore the concept of *Pareto optimality* (or *non-dominance*) in multi-objective decision making. In a problem with two objectives, solution x dominates solution y if $x_1 > y_1$ **and** $x_2 > y_2$ (assuming a maximization problem, where larger values are preferred for both objectives). The same definition holds in higher dimensions: x dominates y if x is preferred in **all** objectives. The set of optimal solutions in a multi-objective problem are those that are non-dominated with respect to all other candidate solutions. Consider the following example:

   A town is situated by a shallow lake. The town would like to increase their economic development through industrial activity that discharges pollution into the lake. The town has to decide how to balance multiple objectives related to phosphorous pollution, economic activity, and the potential for irreversible environmental damages.



Part a: A Three-Objective View of the Pareto Surface

On ETL, we have uploaded a csv: Lake_Problem_Pareto.csv. These are pareto optimal solutions for this 4-objective environmental management problem. The four columns in the file represent the four objectives, while the 262 rows represent candidate solutions. The four objectives are: maximize economic benefit (column 1), minimize the phosphorous in the lake (column 2), maximize the probability of avoiding rapid pollution policy changes termed—inertia (column 3), and maximize reliability of permanently degrading the lake's water quality (column 4). The last three columns will be used later in part d.

Using J3, import the data and **plot the pareto surface with a 3D plot**. Using the plot options button, put inertia, economic benefits, phosphorus, and reliability on the x, y, z, and color axes respectively. Using the widgets button, add a colorbar. Save your figure (using the camera button) and mark the **preferred direction for each objective**, as well as **the**

ideal point on the graph.

Note: the values for the phosphorus objective have a negative sign in front, please ignore this and assume absolute values (e.g., phosphorus |-1.2| is worse than phosphorus |-0.5|)

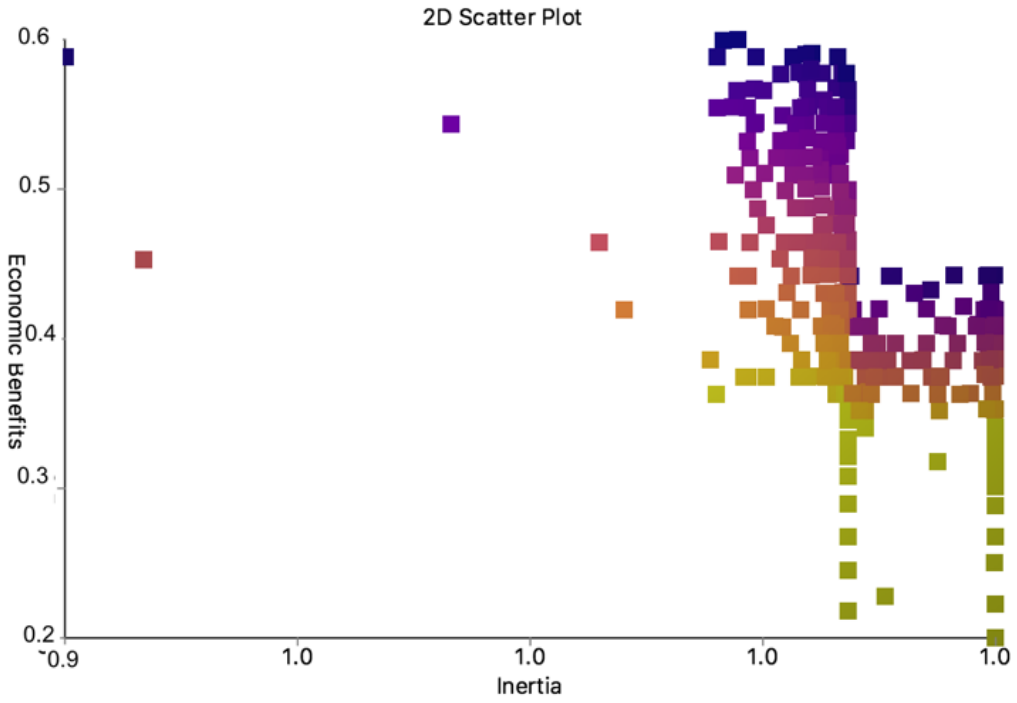
Part b: Examining Pareto optimality of solutions

Using the widgets button we can generate two-dimensional scatter plots with the following axes:

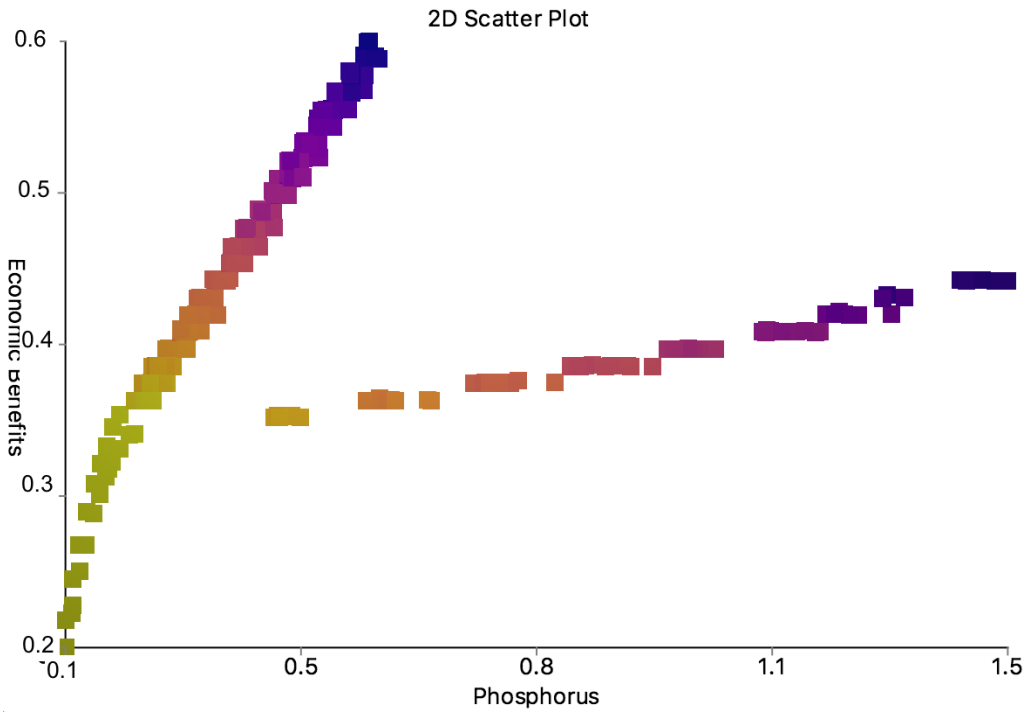
i) Economic benefits on the Y-axis and Inertia on the X-axis
ii) Economic benefits on the Y-axis and Phosphorus on the X-axis
iii) Reliability on the Y-axis and Phosphorus on the X-axis

You can try this but it's not necessary to get full marks. The three plots are given below:

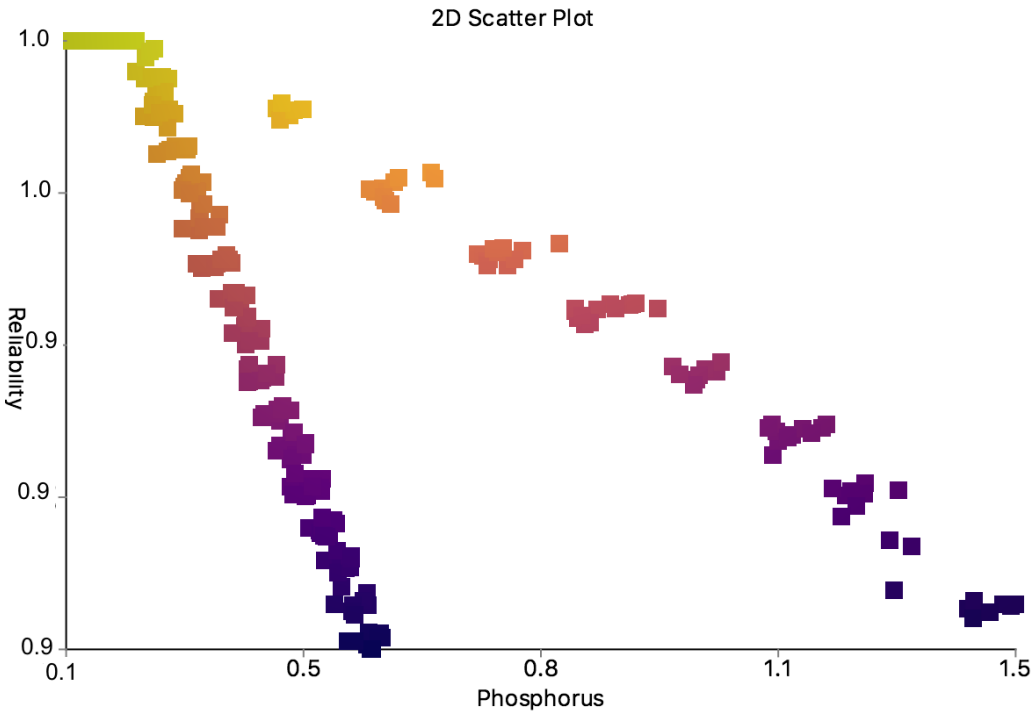
i)



ii)



iii)

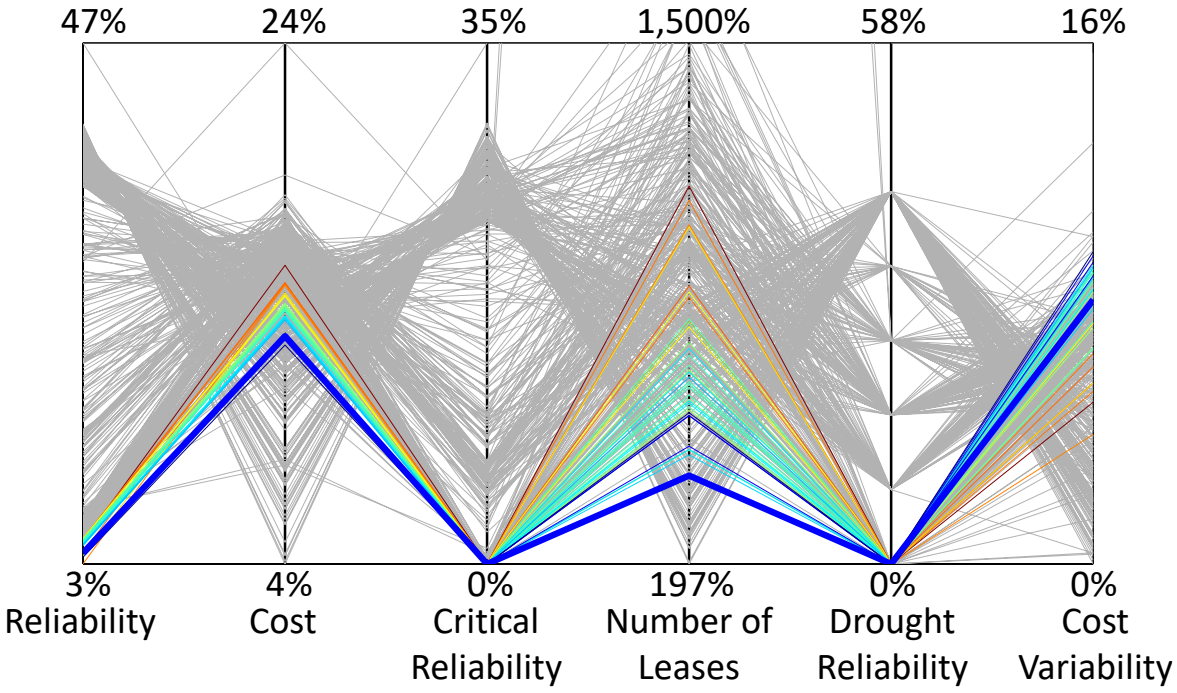


Assuming we only care about the two objectives plotted in each plot, mark the ideal point in each and highlight the Pareto front.

Part c: Visualizing Tradeoffs with a Parallel Axis Plot

Multi-objective optimization problems commonly have more than three dimensions. The Lake Problem data that you were given with this assignment is actually the result of a four-objective optimization—that is, in its four-objective space, all solutions should be non-dominated.

One method to visualize sets of solutions in more than three objectives is the *parallel axis plot*. Here's an example for a six-objective problem: (Kasprzyk et al. 2013 *Env. Mod. Soft.*)



The objectives are arranged along the x-axis, and the y-axis represents their values. Each line across the plot represents a single solution, and where it crosses each "parallel axis" represents its value in that objective. In this example, the plot also uses "brushing", i.e. graying out solutions that do not meet certain criteria.

Using the widget button, **make a parallel axis plot** of the Lake Problem data showing all 4 objectives. Save the plot **and draw a line where the ideal solution would be**. **According to the plot, what are the tradeoffs (which objectives are in conflict)? What general conclusions can you make about the behavior of the objectives?**

Part d: Choosing a Potential Solution

You meet with the town elders and it becomes clear to you that they prefer a solution that will keep their economic benefits above 0.4 but also keep the phosphorus in the lake below 0.5. **Find a solution that meets these requirements and mark it on your parallel axis plot.** In order to do this, use the "brushing" tool, found in the widgets, and limit the ranges of the objectives accordingly. Then, pick a solution of your choice and double click to mark it. **Why did you pick this point (there's no wrong answer here, just explain why one would potentially prefer the solution you chose)? Report the values for inertia, economic benefits, reliability, and phosphorus.** To find how the solution you picked performs, use the "display details" tool in the widgets and click on your selected solution in the 3D scatter view.

Part e: Analyzing Robustness

When formulating your model, you made some assumptions about initial parameters in the lake, because you are a critical thinker and are taking a decision analysis class, you are a little uncertain about the how the solutions you chose will behave if these assumptions prove incorrect. While the solution meets the requirements of the decision makers, you are concerned that if you change the value of some of the initial parameters, your results may change. You structure a robustness analysis as follows: you come up with a plausible range for the uncertain parameters and you sample uniformly, creating 1000 alternate states of the world. You run your decision variables through the lake model to see how your objective values have changed. In Lake_Problem_Pareto_Robustness.csv, locate your marked solution and report the values in the three columns after the objective values. These values are the percent of states of the world (as a decimal) that your solution was able to meet certain requirements. **Fill out the table with these values.**

Requirements	Percent of SOWs
Average Economic Benefits > 0.2	
Average Reliability > 95%	
Average Economic Benefits > 0.2 & Average Reliability > 95%	

What are the implications of these results? How confident are you in the original solution that you picked?

Download instructions for J3

To download J3, go to https://github.com/Project-Platypus/J3#get-it.

Pre-packaged distributions are available for Windows, Linux, and Mac. All distributions include sample data files you can test.

Windows

Download and extract J3-Win.zip if you already have Java 8 installed. Otherwise, download J3-Win-JRE.zip, which is bundled with the Java 8 runtime environment. After extracting, run J3.exe. You can also load a data set from the command line by running J3.exe <file>.

You can also download and run J3.exe by itself, although you will need to provide your own data files.

Linux (Debian, Ubuntu, etc.)

A deb file is provided to assist installing J3 on Linux. This installation requires openjdk-8-jre. On Ubuntu, we needed to add the following repository to satisfy this dependency:

sudo apt-add-repository ppa:openjdk-r/ppa

sudo apt-get update

On some versions of Linux, JavaFX may not be bundled with OpenJDK. If this is the case, run sudo apt-get install openjfx to install JavaFX.

Finally, download and install J3-1.0-1.deb. After installation, J3 will appear as a desktop application. You can also launch the program by running the command J3.

Mac

Download and extract J3-Mac.zip if you already have Java 8 installed. Otherwise, download J3-Mac-JRE.zip, which is bundled with the Java 8 runtime environment. After extracting, run J3.app.